Residual connections is one of the reason why deep neural network can be trained.

problems in standard model training

$$y = f_n(\cdots f_3(f_2(f_1(x;\theta_1);\theta_2);\theta_3)\cdots;\theta_n)$$

Observations:
in AlexNet, when $n \uparrow$, i.e. the network becomes deeper,

both traning and validation loss $\uparrow$.

implication: the problem is not the generalization of deeper model,
(otherwise we should see traing loss $\downarrow$, validation loss $\uparrow$)
but the traing itself.

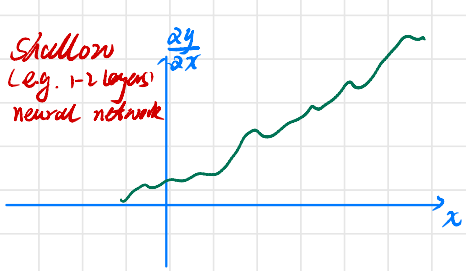
possible explanation: Shattered gradients

in standard model training, we have

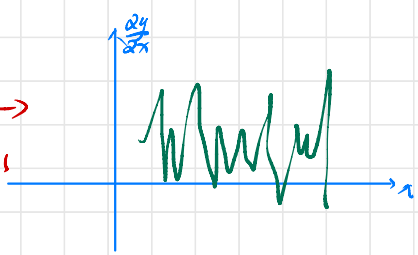
$$\frac{\partial y}{\partial f_1} = \frac{\partial y}{\partial f_n}\frac{\partial f_n}{\partial f_{n-1}}\cdots\frac{\partial f_2}{\partial f_1}$$

→ when the model goes deeper and deeper, a small change in x leads to very different gradients. So the loss surface is changing all the time with different batches...

Consider univariate input x → univariate input y



Shattered gradients in deep neural networks →



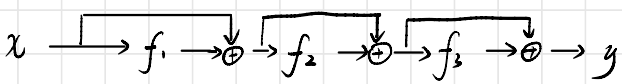
Residual connection: Consider $n = 3$

$$\lambda_1(x) = x + f_1(x; \phi_1)$$

$$\lambda_2(x) = \lambda_1 + f_2(\lambda_1; \phi_2)$$

$$y = \lambda_2 + f_3(\lambda_2; \phi_3)$$

So $y = x + f_1(x; \phi_1) + f_2(x + f_1(x; \phi_1); \phi_2) + f_3(x + f_1(x; \phi_1) + f_2(x + f_1(x; \phi_1); \phi_2); \phi_3)$



Implications of the formula:

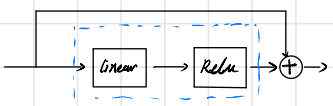
1. the lower level information is kept
2. we can think residual connections as an ensemble of neural network
3. there are many different paths to pass information from input to output

Implementation of residual networks

There are two things to be careful when implement residual network

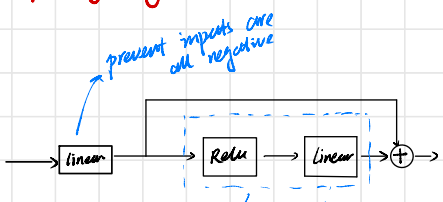
1. order of operations
2. exploding gradients

Order of operations:



the most common layer we will use

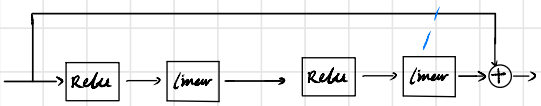
problem: Can only add positive values



Swap Relu and Linear

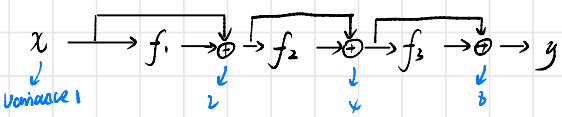
Lead to

Both ends with Linear



in practice, a residual block could have multiple layers.

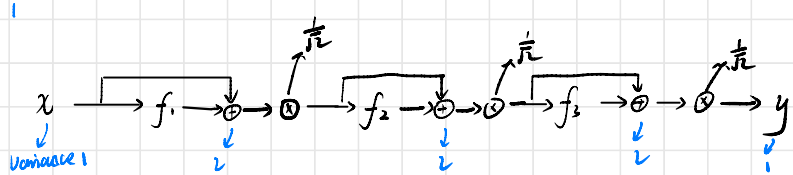
exploding gradients



Variance grows exponentially

backward is the same

Solution 1

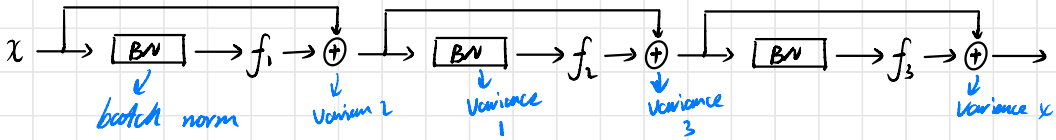


$$y = \left(\frac{1}{\sqrt{2}}\right)^3 x + f_1(x;\phi_1)$$
$$+ \left(\frac{1}{\sqrt{2}}\right)^2 \left(f_2\left(x + f_1(x;\phi_1);\phi_2\right)\right)$$
$$+ \frac{1}{\sqrt{2}}\left(f_3\left(x + f_1(x;\phi_1) + f_2(x + f_1(x;\phi_1);\phi_2);\phi_3\right)\right)$$

This is not good. although the variance is stable, but the values of lower layer inputs are scaled down.

shattered problem persists

Solution 2:



now the variance increases linearly, not exponentially.

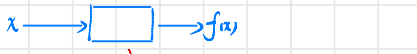
Side effect: the later layer has small effect (Var = 1) to the residual connections (Var = K) So at the beginning of train, the network is effectively shallower.

Why does residual connection work?

There are many different explanations:

1. the work for each layer is easier...

Before

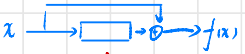


this layer need to learn the function f

↓

each layer needs to consider how to transform to approach outputs

After



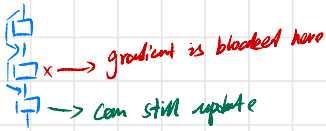
this layer need to learn f(x) - x

↓

each layer "adds" information to current solutions to approach final outputs

2. the shattered gradient problem are much less prominent and the loss landscape is much smoother

lower layer input contributes to outputs directly → much easier to take their gradients



3. an ensemble of many small model

4. many different paths, there may be one has good prior (winning lottery theory)